

Internship progress report

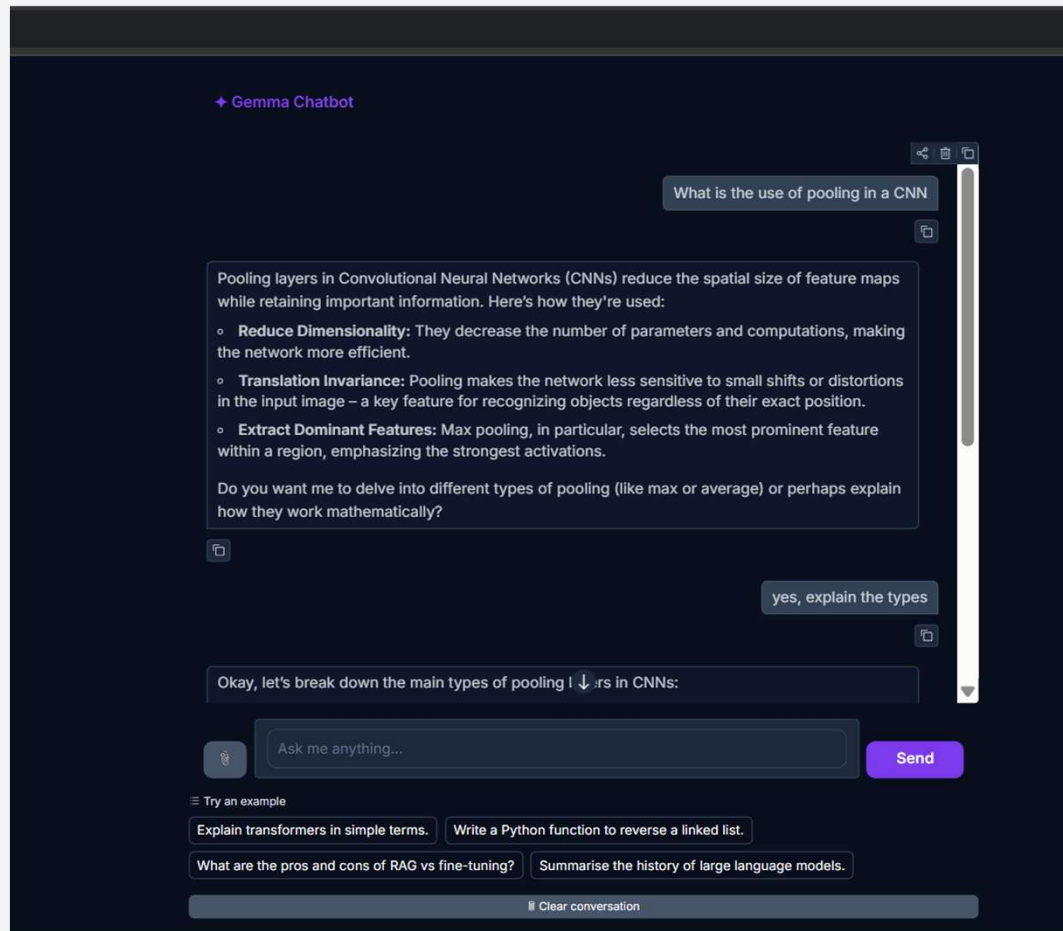
LLM Chatbot and LLM+RAG pipeline

By S Sriram

Plain Gemma Chatbot Architecture

- **Model Selection:** Powered entirely by the Gemma 3 4B (gemma3:4b) variant via Ollama. This choice strikes an optimal balance between low-latency local execution and robust semantic synthesis on local hardware.
- **Execution Flow:**
 - **Initialization:** Instantiates a minimal Gradio interface wrapped in a custom corporate-slate theme.
 - **State Management:** Conversational history is parsed dynamically and structured into formal `{"role": "user/assistant", "content": "text"}` object arrays.
 - **Document Parsing Helper:** Features an integrated PdfReader extraction block that sanitizes local .pdf, .txt, and .docx files, truncating contexts to a safe 12,000 character buffer.
 - **Payload Assembly:** Injects a strict system prompt targeting clear, non-hallucinatory interactions, merging active history with the text or image-base64 streams.
 - **Streaming Execution:** Dispatches an asynchronous POST request to the local Ollama API endpoint (/api/chat), incrementally yielding tokens directly into the UI frontend.

Gemma Chatbot Conversation Example



↑ Gemma Chatbot

What is the use of pooling in a CNN

Pooling layers in Convolutional Neural Networks (CNNs) reduce the spatial size of feature maps while retaining important information. Here's how they're used:

- **Reduce Dimensionality:** They decrease the number of parameters and computations, making the network more efficient.
- **Translation Invariance:** Pooling makes the network less sensitive to small shifts or distortions in the input image – a key feature for recognizing objects regardless of their exact position.
- **Extract Dominant Features:** Max pooling, in particular, selects the most prominent feature within a region, emphasizing the strongest activations.

Do you want me to delve into different types of pooling (like max or average) or perhaps explain how they work mathematically?

yes, explain the types

Okay, let's break down the main types of pooling ↓ ↓ ↓s in CNNs:

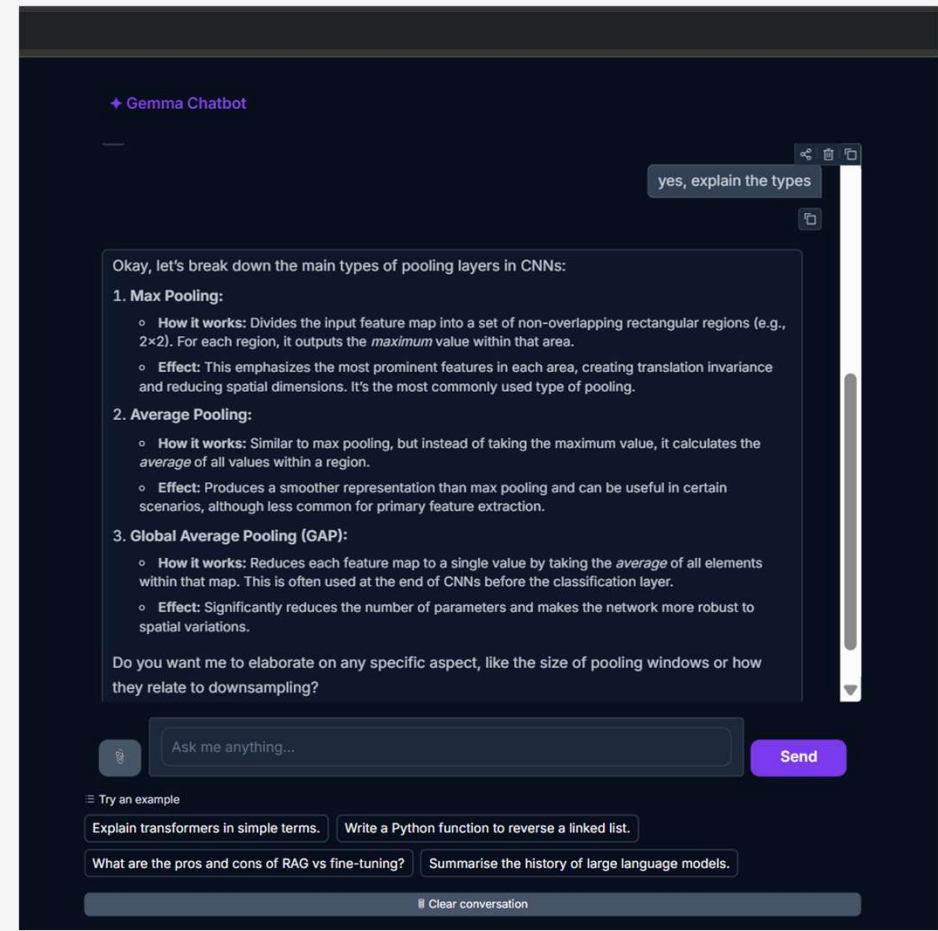
Ask me anything... Send

Try an example

Explain transformers in simple terms. Write a Python function to reverse a linked list.

What are the pros and cons of RAG vs fine-tuning? Summarise the history of large language models.

Clear conversation



↑ Gemma Chatbot

yes, explain the types

Okay, let's break down the main types of pooling layers in CNNs:

- 1. Max Pooling:**
 - **How it works:** Divides the input feature map into a set of non-overlapping rectangular regions (e.g., 2×2). For each region, it outputs the *maximum* value within that area.
 - **Effect:** This emphasizes the most prominent features in each area, creating translation invariance and reducing spatial dimensions. It's the most commonly used type of pooling.
- 2. Average Pooling:**
 - **How it works:** Similar to max pooling, but instead of taking the maximum value, it calculates the *average* of all values within a region.
 - **Effect:** Produces a smoother representation than max pooling and can be useful in certain scenarios, although less common for primary feature extraction.
- 3. Global Average Pooling (GAP):**
 - **How it works:** Reduces each feature map to a single value by taking the *average* of all elements within that map. This is often used at the end of CNNs before the classification layer.
 - **Effect:** Significantly reduces the number of parameters and makes the network more robust to spatial variations.

Do you want me to elaborate on any specific aspect, like the size of pooling windows or how they relate to downsampling?

Ask me anything... Send

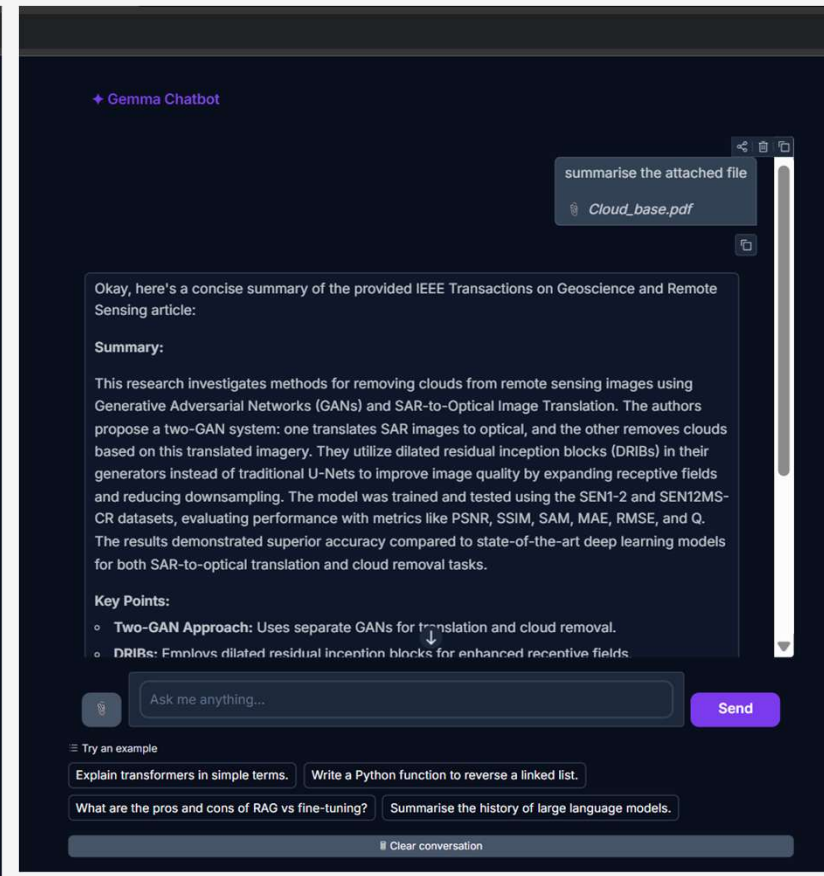
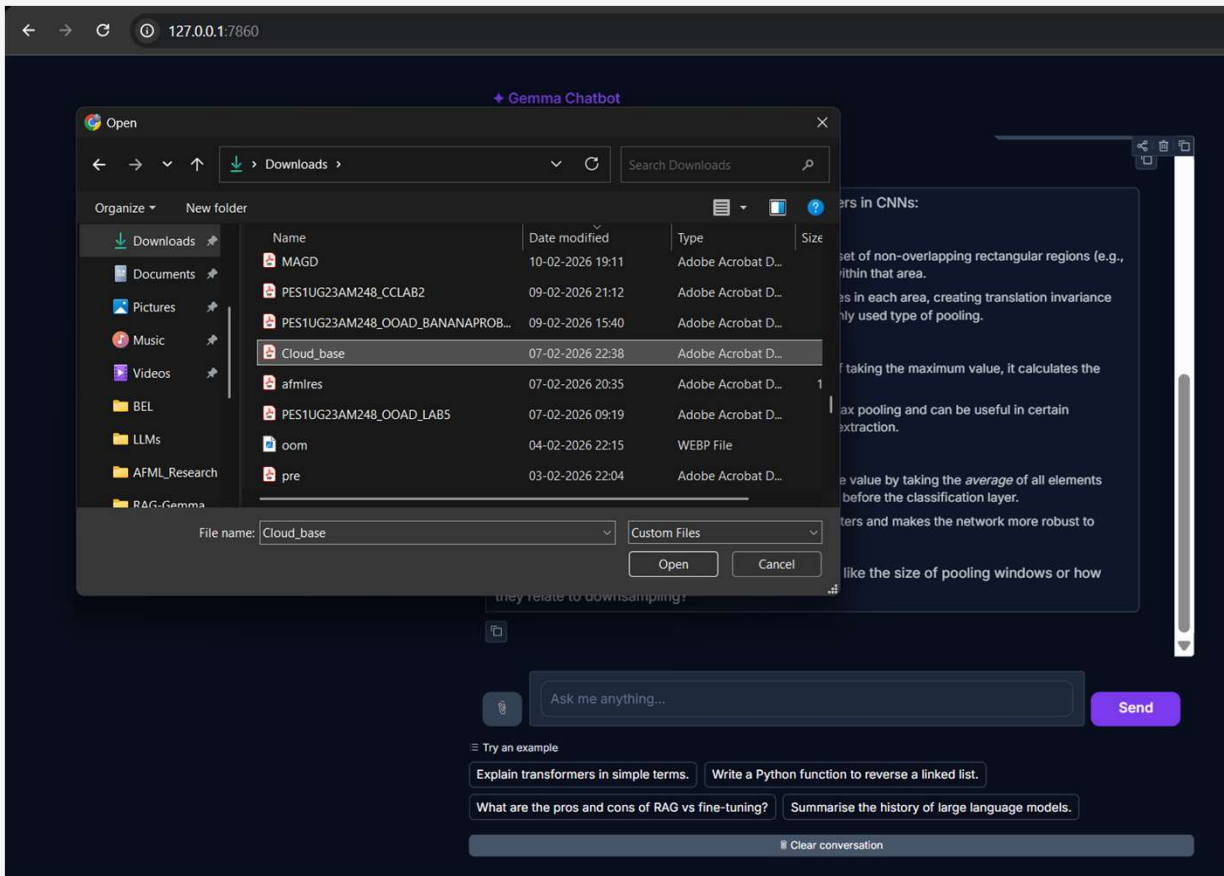
Try an example

Explain transformers in simple terms. Write a Python function to reverse a linked list.

What are the pros and cons of RAG vs fine-tuning? Summarise the history of large language models.

Clear conversation

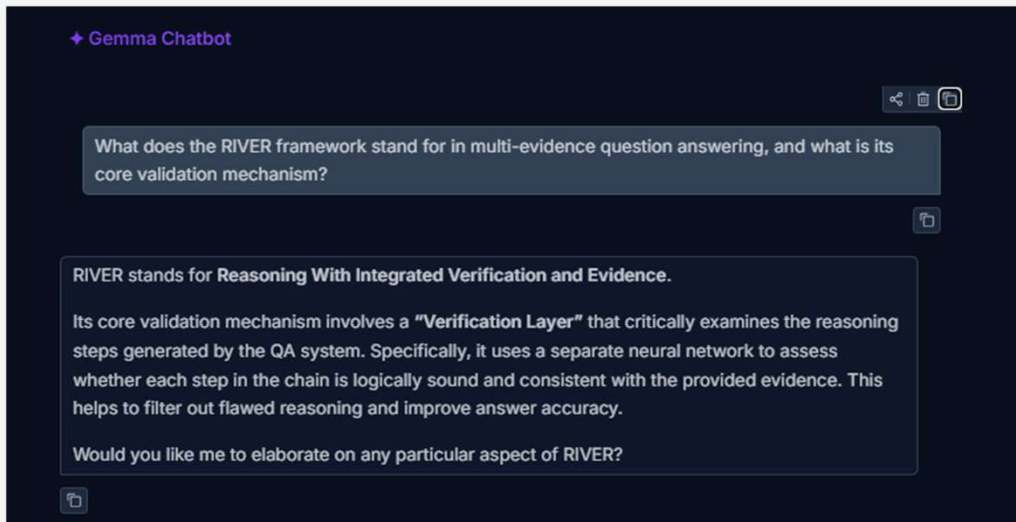
In-Context File Injection Interaction



RAG Pipeline Infrastructure

- Target Knowledge Base: Direct API-driven compilation of 171 unique, information-dense cs.CL and cs.AI arXiv paper abstracts published between March 2025 and June 2025 that Gemma 3 4B has not encountered in training.
- Ingestion Flow:
 - Strips raw LaTeX symbols and heavy whitespace parameters down to clean prose blocks.
 - Implements a sliding character window chunker configured to a `CHUNK_SIZE` of 512 and `CHUNK_OVERLAP` of 64.
 - Computes dense vector embeddings locally using the 384-dimensional all-MiniLM-L6-v2 SentenceTransformer architecture.
 - Persists historical vectors and indexing attributes into a local, isolated ChromaDB collection.
- Application Flow:
 - Generates a 384-dimensional text embedding for the live incoming user query.
 - Queries ChromaDB to isolate the `TOP_K = 5` closest vector neighbors from the 2025 paper slice.
 - Formats these fragments into an explicit 'Retrieved Context' prompt layer.
 - Overrides default behavior with an authoritative teacher-style system prompt instructing the model to synthesize the provided background documentation.

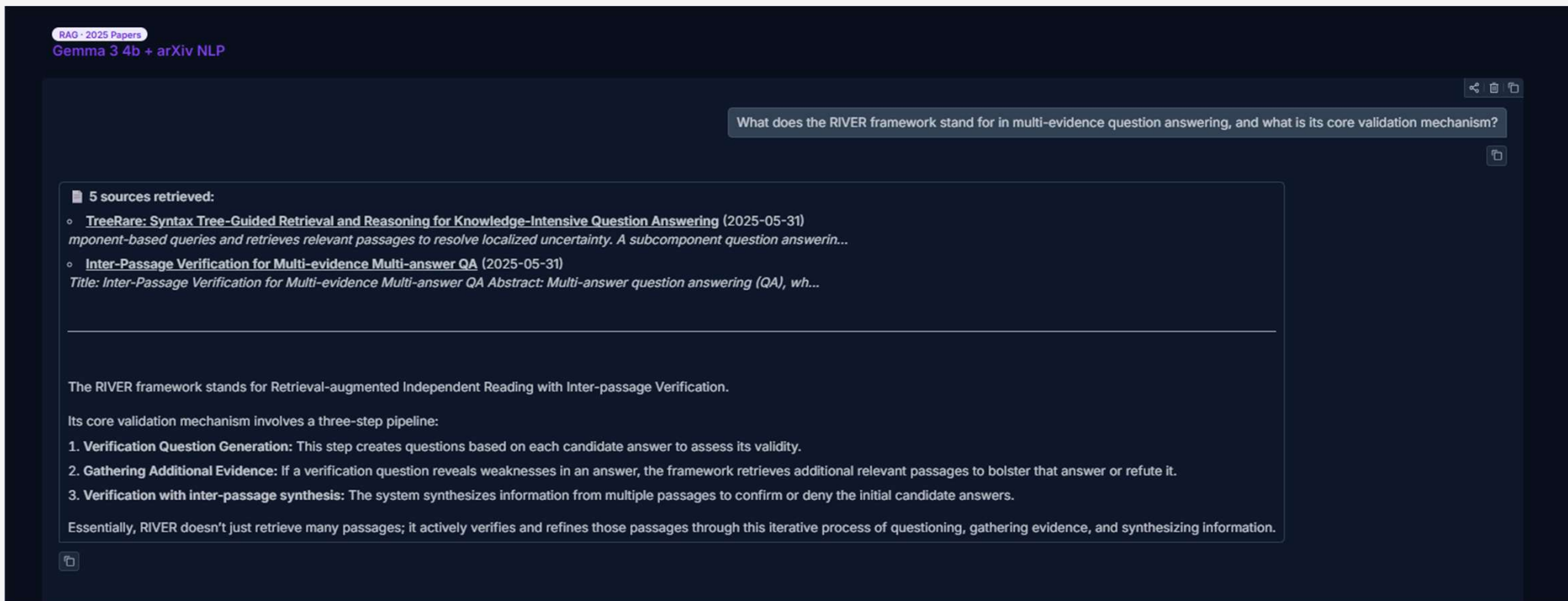
Example 1 — Gemma+Rag vs Gemma Chatbot



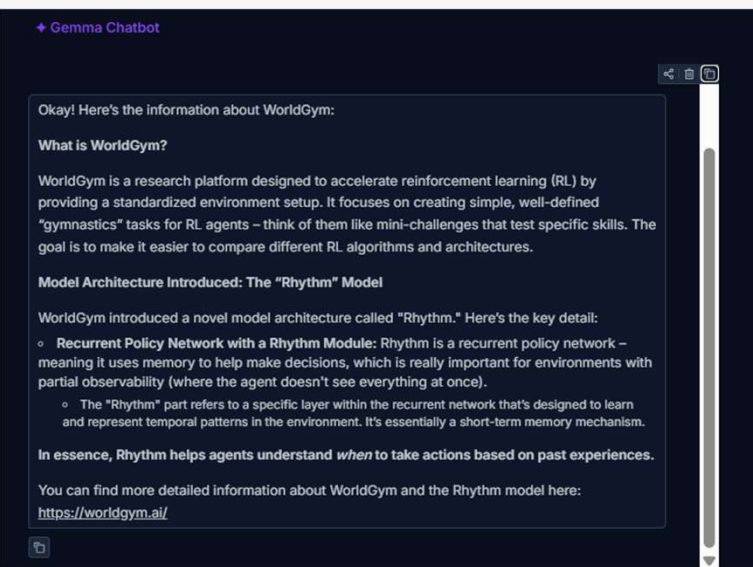
Evaluation Insights:

Base Model Failure: The base chatbot experiences absolute semantic drifting. Lacking context, it invents a highly plausible-sounding but totally fabricated title: 'Reasoning With Integrated Verification and Evidence.'

RAG Resolution: By anchoring the prompt to real-time episodic lookups, the system cleanly isolates the true technical definition: Retrieval-augmented Independent Reading with Inter-passage Verification. It lists the authentic 3-step validation pipeline.



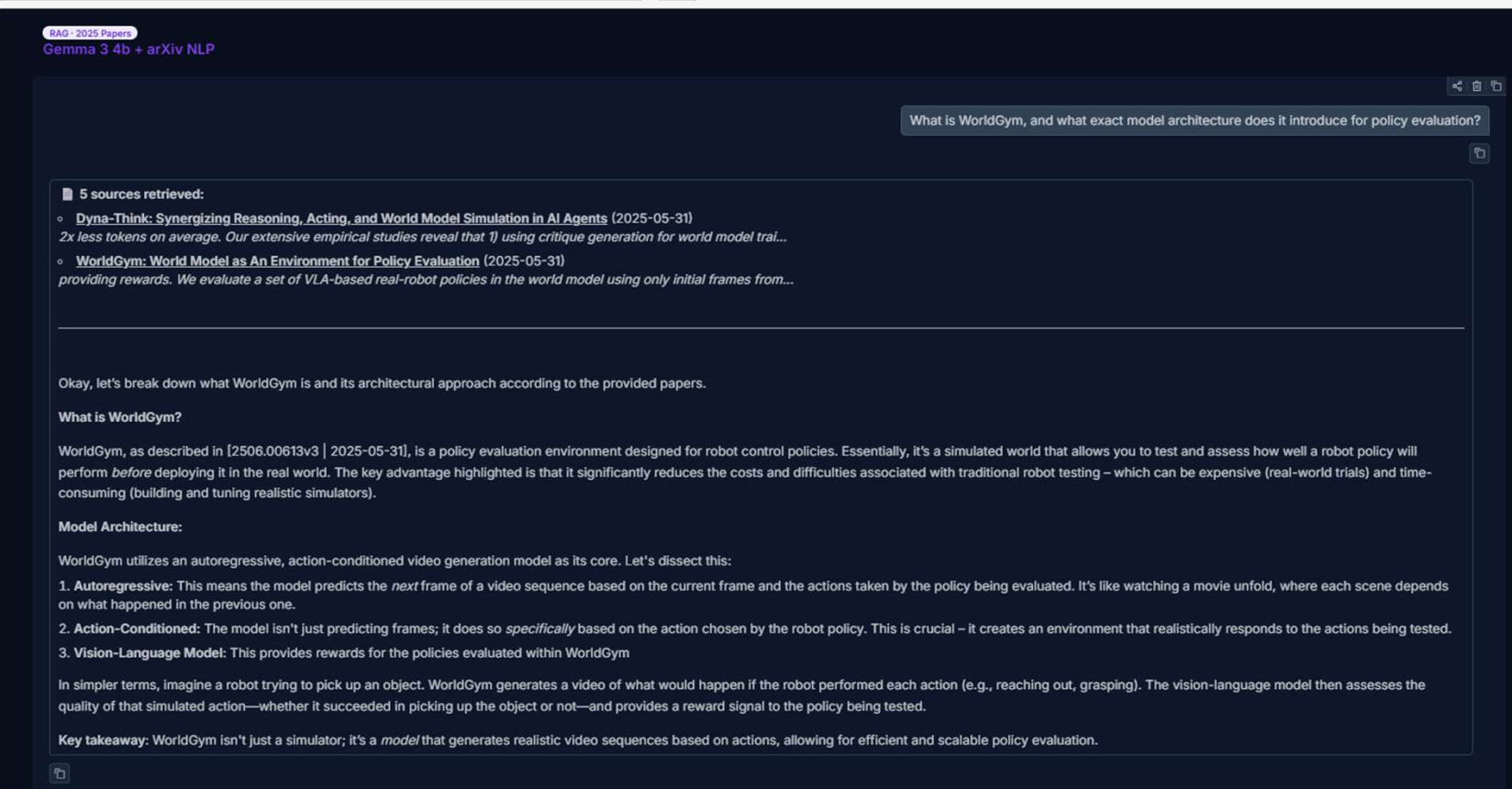
Example 2 — Gemma+Rag vs Gemma Chatbot



Evaluation Insights:

Base Model Failure: The base model undergoes a complete logical hallucination. It translates the keyword 'Gym' into physical human fitness challenges and conceptualizes a fictional 'Rhythm Model' to back it up.

RAG Resolution: The RAG framework anchors the query into late-May 2025 data, precisely mapping WorldGym to its true purpose as a simulated proxy world for physical robot control policies using an autoregressive, action-conditioned video generator.



Example 3 — Gemma+Rag vs Gemma Chatbot

Gemma Chatbot

In noisy in-context learning, what is a Sample Cleanliness Score, and how is it derived?

Okay, let's break down the Sample Cleanliness Score (SCS) used in Noisy In-Context Learning!

What is it?

The Sample Cleanliness Score (SCS) is a metric that estimates how "pure" or "clean" a set of examples is for a language model during in-context learning. Essentially, it measures whether the examples are genuinely representative of the desired task and not contaminated by irrelevant information.

How is it derived?

Here's the general process:

1. **Generate Examples:** The model generates several example prompts and their corresponding answers (the "in-context" examples).

2. **Calculate Perplexity:** For each example, the SCS calculates the perplexity of that example given only the other examples in the batch. Perplexity is a measure of how well the language model predicts the next token in a sequence – lower perplexity means better prediction.

3. **Aggregate Perplexities:** The perplexities from all the examples are then averaged or aggregated to create a single SCS score for that batch of examples.

In simpler terms: If an example has a low perplexity when evaluated only against the other in-context examples, it's considered clean and contributes positively to the SCS. A high perplexity suggests the example is noisy and detracts from the SCS.

Do you want me to elaborate on any specific part of this, like:

- The math behind perplexity?
- How SCS relates to model performance in Noisy In-Context Learning?

Evaluation Insights:

Base Model Failure: The baseline chatbot outputs a highly dangerous 'expert trap' error. It generates a perfectly coherent, well-structured, but entirely fictitious mathematical definition regarding basic batch-perplexity averaging.

RAG Resolution: The RAG-grounded response completely bypasses this instinctual guessing. It pulls exact methodology from the Dual Debiasing framework, noting that the score is rigorously derived using synthesized neighbors to isolate biases.

RAG: 2025 Papers
Gemma 3 4b + arXiv NLP

In noisy in-context learning, what is a Sample Cleanliness Score, and how is it derived?

5 sources retrieved:

- [Dual Debiasing for Noisy In-Context Learning for Text Generation](#) (2025-05-31)
nnnotations, highlighting two sources of bias in perplexity: the annotation itself and the domain specific knowledge inhe...
- [PMF-CEC: Phoneme-augmented Multimodal Fusion for Context-aware ASR Error Correction with Error-specific Selective Decoding](#) (2025-05-31)
method outperforms other contextual biasing methods, and remains valuable compared with LLM-based methods in terms of fa...
- [Understanding Behavioral Metric Learning: A Large-Scale Study on Distracting Reinforcement Learning Environments](#) (2025-05-31)
A key approach to state abstraction is approximating behavioral metrics (notably, bisimulation metrics) in the observati...

According to Passage 1 and 2 of "[2506.00418v2 | 2025-05-31] Dual Debiasing for Noisy In-Context Learning for Text Generation", the Sample Cleanliness Score is a metric that uncovers absolute sample cleanliness regardless of the overall corpus noise level.

It's derived through a dual debiasing framework which uses synthesized neighbors to explicitly correct perplexity estimates. Essentially, the system identifies and corrects biases within perplexity measurements – these biases come from two main sources: the annotation itself and the domain-specific knowledge embedded in large language models (LLMs).

This approach moves beyond simply ranking perplexities based on the assumption that noisy samples will have higher values. It's a more robust method for detecting noise because it actively corrects for these inherent biases.